WikiPulse: Real-Time Analysis of Wikipedia Edit Streams

ID2221 final project · Muyang Huang, Yanjun Wang, Hao Tang · code, run instructions and raw results in this repository

1. What we built

WikiPulse ingests the Wikimedia `recentchange` feed end to end, from the raw stream to a live dashboard. We run the whole platform as one Docker Compose stack on a laptop: Kafka 3.9, HDFS 3.2, Spark 4.2 with Delta 4.4, Cassandra 4.1 and Grafana 12.2.

A Python producer reads the SSE feed and writes every raw event to Kafka, keyed by the Wikimedia `meta.id`. It saves its SSE position only after Kafka has acknowledged the write, so delivery is at least once. From Kafka the stream goes two ways. One Structured Streaming job archives it unfiltered to an HDFS Delta table, with payload, headers and offsets. Our analysis job applies the shared rules in `events.py`: it keeps Wikipedia articles with `type` edit or new, removes repeated source IDs and uses a 2-minute event-time watermark. It then runs three append-mode queries. The first ranks trending pages in 10-minute windows sliding by 1 minute, scored `edit_count·ln(1+editors)`. The second flags edit-war candidates, pages with at least 3 non-bot reverts by at least 2 users and an A→other→A alternation within 30 minutes. The third counts 5-minute activity, split into bots and humans, for every edition and treats a z-score above 3 against the baseline as an anomaly. Results go to Cassandra tables partitioned by `(wiki, day)` or `(day)`. A Grafana dashboard reads them through the Cassandra plugin and shows trending pages, edit-war candidates, the bot/human ratio and share, language share, baseline and activity heatmaps, and anomalies.

Two decisions shaped the rest of the design. We made the baseline a batch product: a Spark SQL job computes the same 5-minute counts from one pinned Delta version and publishes the mean and standard deviation per (wiki, UTC hour of day) as an immutable version. The stream reads one fixed version and moves to a new one only on a controlled restart, so a running job never sees its baseline change underneath it. We also had to work around a missing piece. No Cassandra connector supports Spark 4, so our `foreachBatch` collects each completed micro-batch on the driver and writes it with prepared INSERTs. Every write overwrites by primary key, which makes a retried batch idempotent.

2. Data

Our live capture had stopped after the first 8-hour handoff, and a baseline needs days of history. EventStreams keeps about a week, so we rebuilt seven days from it, resuming from a `Last-Event-ID` positioned by timestamp for each data centre. Once we grouped the Kafka acknowledgements, the backfill ran at about 1,800 events/s instead of about 30. The pinned archive (Delta version 348) holds 20.4 M raw events from 2026-10-17 12:00 to 10-24 18:15 UTC, or 2.9 M per day, and its row count equals the Kafka offsets exactly. Wikipedia projects make up 28.9% of the raw stream. Article edits and new pages, which are what we analyse, are 11.9% (2.43 M rows from 335 editions). Baseline `20261023T1200Z` covers six days with no collection gap, which gives 72 samples for every (wiki, hour) cell.

The backfill had one trap. It is stored in fetch order, not event order, and the sparse codfw stream ran days ahead of eqiad. Replayed as stored, the first future codfw event pushes the watermark past everything after it, and the stream drops all later eqiad events as late. We therefore replay backfilled data in event-time order.

3. Evaluation

We ran every experiment on the Compose stack as deployed: one Spark worker with 1 core and 1 GiB, a 768 MiB driver and 8 shuffle partitions. Each replay reads the same pinned slice starting at 2026-10-23 12:00, after the baseline history, so every configuration sees identical input. The natural rate of that slice is about 34 raw events/s.

Throughput and latency. A's replay sends the same 50,000 archive rows into a fresh topic at a fixed rate, and we ran each configuration twice. For latency we wanted the delay the system adds, not the delay the window definition imposes. We therefore start the clock at the Kafka append time of the first event whose event time passes `window_end + 2 min`, the moment the window could close, and stop it at the window's `written_at` in Cassandra. That covers trigger wait and processing and leaves out the watermark wait.

Kafka partitions	Rate (events/s)	Trigger	Hot pages latency p50 / p95	Activity latency p50
3	100 / 200 / 400	5 s	17 / 26 · 18 / 31 · 17 / 22 s	28 · 37 · 27 s
3	100 / 200 / 400	30 s	48 / 60 · 51 / 64 · 54 / 64 s	47 · 48 · 45 s
3 vs 1	200	10 s	19 / 25 vs 19 / 24 s	25 vs 15 s

Every paced configuration kept up with its input. Throughput equalled the offered rate, and the last batch finished within 34 s of the end of the replay. To find the ceiling we preloaded the 50,000 rows and let the job read them in 10,000-row batches; it drained 634 to 737 events/s, about 20 times the natural rate.

The trigger interval decides latency; the input rate barely matters. A window is emitted only by the batch after the one that advanced the watermark, so a 30 s trigger costs about 50 s. At 5 or 10 s the batches already run back to back, because three stateful queries take 7 to 12 s per batch on the single core. Latency then comes to about two batch durations, 17 to

19 s for hot pages. More Kafka partitions did not help either. On one executor core they made no measurable difference, and the spread between repetitions was as large as any partition effect.

The live run had a different bottleneck. The dashboard fell 15 minutes behind within an hour, and we first suspected Spark. The cause was the producer: it waits for every Kafka acknowledgement, which caps it at about 32 events/s, below the evening feed. Grouped acknowledgements (`--ack-every`) remove that limit and still deliver at least once.

Correctness. We recounted a replay topic in batch with the same rules, and it matched every closed `activity_5m` window, 198 of 198 windows and 8,821 edits. In the live run we killed the streaming driver and restarted it from its checkpoint. It resumed with contiguous 5-minute windows and no duplicates. The same run flagged an enwiki revert burst of 151 reverts in 5 minutes against a baseline of about 17 ($z = 16.7$).

Edit-war precision. Over seven days the heuristic flagged 1,052 windows on 209 pages. We labelled a seeded random sample of 50 pages from their edit summaries using B's rubric, under which only a substantive dispute over content or sourcing counts as positive. The sample had 8 positive, 33 negative and 9 uncertain pages, so precision is 19.5% (8/41, Wilson 95% CI 10 to 34%), or 16% if the uncertain pages count as negative. Most false positives are anti-vandalism work that fits the same alternation pattern: patrol-tool reverts (14), vandalism rollbacks (14) and sockpuppet cleanup (5). This is why our dashboard calls these pages candidates. Our next step would be to filter out reverts whose summaries name a tool, vandalism or a block, or that target temporary accounts.

Delta partition pruning. A's archive is partitioned by ingest date, so the whole backfill sits in one partition and pruning has nothing to skip. To test pruning on a fair layout, we copied version 348 into a table partitioned by event date. We then ran the same one-day 5-minute count job three times each way and disabled Delta data skipping, so that only partition pruning differed. Both variants produced identical output.

	Files read	File bytes	Median time
Partition filter on	292	9.4 GB	51.4 s
Event-time range only	4,580	71.3 GB	60.9 s

Pruning skipped 94% of the files but saved only 16% of the time. The unpruned scan already avoids most of the payload through Parquet row-group min/max filtering, because the files are close to event-time order. The rest of the job costs the same however much it scans. Partitioning by event date pays off for scans that cannot be pushed down to row groups, and it keeps planning and file listing proportional to the requested range.

4. Limitations and lessons

One 1-core executor handles about 20 times the natural rate, but it leaves no headroom for larger state. It was also too small for the multi-day batch jobs, so we ran those in local mode on the client container. On this hardware we would choose a 5 or 10 s trigger, which gives about 18 s for trending pages. Longer triggers only add delay, since even a 5 s trigger keeps up at 12 times the natural rate. Our precision estimate rests on one AI-assisted rater reading edit summaries without a second review, and the sample has few non-English pages. Finally, partitioning the raw archive by ingest date suits appending and retention but not historical batch jobs. For baselines we would build from a compacted copy partitioned by event date.